

JAVA PROGRAMMING

Chapter 5: Input & Output in Java

By Akanshu Jamwal

Table of Contents

1. What is Input & Output?
2. Java Output Methods
3. print() vs println()
4. printf() - Formatted Output
5. Printing Variables and Literals
6. String Concatenation
7. Java Input with Scanner
8. Scanner Methods for Different Types
9. next() vs nextLine()
10. Complete I/O Example
11. Why Input/Output Matters
12. Practice Exercise

1. What is Input & Output?

In any programming language, you must know how to display output and take input. These are the foundation of interaction between your program and the user.

In Java, this is handled using:

- `System.out.print()` - Display without newline
- `System.out.println()` - Display with newline
- `System.out.printf()` - Formatted output
- Scanner class - Read user input

2. Java Output Methods

Java sends output to the console using the `System.out` object with different methods.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Welcome to CodeHelp ONE!");  
    }  
}  
Output: Welcome to CodeHelp ONE!
```

- `System` → A built-in class
- `out` → A static object used for output
- `print/println/printf` → Methods to display data

3. `print()` vs `println()`

The key difference is how they handle line breaks:

Method	Behavior	Example
<code>println()</code>	Prints text + moves to next line	Step 1\nStep 2
<code>print()</code>	Prints text + stays on same line	Phase A Phase B

```
System.out.println("Step 1: Learn");
System.out.println("Step 2: Practice");

System.out.print("Phase A ");
System.out.print("Phase B");

Output:
Step 1: Learn
Step 2: Practice
Phase A Phase B
```

4. printf() - Formatted Output

printf() allows formatted printing with placeholders. It's similar to C/C++ printf.

```
String name = "Aman";
int solved = 150;

System.out.printf("Student: %s, Problems: %d", name, solved);
// Output: Student: Aman, Problems: 150
```

Common Format Specifiers:

Specifier	Type	Example
%s	String	%s → Aman
%d	Integer	%d → 150
%f	Float/Double	%f → 4.8
%c	Character	%c → A
%b	Boolean	%b → true

Used for: Reports, Dashboards, Formatted outputs, Aligned columns

5. Printing Variables and Literals

When printing numbers or variables, do NOT use quotation marks. Quotes treat content as text, not values.

```
int activeUsers = 2500;
double rating = 4.8;

System.out.println(activeUsers);    // Correct - prints 2500
System.out.println(rating);         // Correct - prints 4.8

System.out.println("activeUsers"); // Wrong - prints text
```

Key Rule: Variables without quotes → prints VALUE | Variables with quotes → prints TEXT

6. String Concatenation

You can combine strings using the + operator. The + operator works differently for strings and numbers.

```
String name = "Riya";
int contests = 12;

System.out.println("Hello " + name);
System.out.println("Contests: " + contests);

Output:
Hello Riya
Contests: 12
```

How + works: Addition for numbers | Concatenation for strings

7. Java Input with Scanner

Java uses the Scanner class to read input from the user. You need to import it first.

Step 1: Import Scanner

```
import java.util.Scanner;
```

This allows your program to use the Scanner class.

Step 2: Create Scanner Object

```
Scanner sc = new Scanner(System.in);
```

- System.in → standard input (keyboard)
- sc → object name (you can name it anything)
- new → creates a new object

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter problems solved: ");
        int problems = sc.nextInt();

        System.out.println("You solved " + problems);
        sc.close();
    }
}
```

Important: sc.close() closes the scanner (good practice to free resources)

8. Scanner Methods for Different Types

Scanner provides different methods to read different data types:

Method	Data Type	Example
nextInt()	Integer	int age = sc.nextInt();
nextDouble()	Double	double salary = sc.nextDouble();
nextFloat()	Float	float gpa = sc.nextFloat();
nextBoolean()	Boolean	boolean active = sc.nextBoolean();
next()	Single Word	String name = sc.next();
nextLine()	Entire Line	String address = sc.nextLine();

Example with Multiple Input Types:

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter accuracy: ");
float accuracy = sc.nextFloat();

System.out.print("Enter platform: ");
String platform = sc.next();

System.out.println("Accuracy: " + accuracy);
System.out.println("Platform: " + platform);

sc.close();
```

9. next() vs nextLine()

This is a commonly tested concept in interviews. Understanding the difference is critical.

Method	Behavior	Example
next()	Reads single word (stops at space)	Input: John Doe\nReads: John
nextLine()	Reads entire line (including spaces)	Input: John Doe\nReads: John Doe

■■ **Interview Alert:** This difference is frequently asked! Using next() when you need nextLine() (or vice versa) causes common input errors.

10. Complete I/O Example

A comprehensive example combining input and output:

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Problems solved this week: ");
        int solved = sc.nextInt();

        System.out.print("Weekly target: ");
        int target = sc.nextInt();

        // Processing
        int remaining = target - solved;

        // Output
        System.out.printf("\nSummary:\n");
        System.out.printf("Name: %s\n", name);
        System.out.printf("Solved: %d\n", solved);
        System.out.printf("Remaining: %d\n", remaining);

        sc.close();
    }
}
```


11. Why Input/Output Matters

In coding interviews:

- Most problems require reading input from the user
- Competitive programming relies heavily on input parsing
- Backend systems process user input continuously
- Output formatting affects test case validation

Strong I/O understanding prevents:

- Input mismatch errors (wrong data type)
- Type conversion errors
- Unexpected crashes due to invalid input
- Wrong output format leading to failed test cases

12. Practice Exercise

⇒ ■ **Build a simple program that:**

1. Takes user's name as input
2. Takes number of problems solved
3. Takes weekly target as input
4. Calculates remaining problems (target - solved)
5. Prints formatted summary using printf()

Expected Output Format:

Name: Aman

Problems Solved: 320

Weekly Target: 350

Remaining: 30

Run it and observe the behavior with different inputs!

What's Next?

Now that you understand input and output, the next logical topics are:

- Conditional Statements (if-else, switch)
- Loops (for, while, do-while)
- Type Casting (converting between types)
- Arrays and Collections

These form the foundation of problem solving. Master I/O handling early to avoid common mistakes in coding interviews!